

MIDAS GUI — Implementation Details (internal)

MIDAS-GUI — Implementation Details (internal)

Scope & status. This is a deep-dive companion to `gui_documentation.md`. It explains *how* the non-obvious techniques are actually implemented in this codebase — the algorithms, the exact parameters and defaults, the formulas, and the reasoning behind design choices — so you don't need any other reference to understand them. It is deliberately kept **out of git** (`.gitignore`) and is not published with the repo.

Everything below is grounded in the current source. Key files: `midas_gui/workers.py` (all heavy compute), `midas_gui/calib.py` (calibration pipeline layer), `midas_gui/widgets.py` (correction toggles, viewers). The GUI only orchestrates; the actual numerics live in the MIDAS backend packages `midas_calibrate_v2` and `midas_integrate_v2`, which the GUI calls.

Table of contents

1. [Detector geometry & the \$\(\eta, R\)\$ cake — the shared foundation](#)
 2. [Bad-pixel masking](#)
 - 2.1 [Statistical auto mask \(spatial outlier + temporal constancy\)](#)
 - 2.2 [Spatial spike rejection](#)
 - 2.3 [Cosmic-ray rejection](#)
 - 2.4 [Spatial spike vs. spatial outlier — the difference](#)
 - 2.5 [Calibration-based masks \(azimuthal \$\sigma\$ -clip, learnable mask\)](#)
 3. [Calibration pipelines](#)
 4. [Refinement with \$\eta\$ -uniformity](#)
 5. [Azimuthal averaging methods & binning kernels](#)
 6. [Physics corrections \(polarization, solid angle\)](#)
 7. [Monitor normalisation](#)
 8. [Drift correction](#)
-

1. Detector geometry & the (η , R) cake

Most techniques below operate on, or produce, the same intermediate object: the **(η , R) cake**, a 2-D polar re-binning of the detector image. Understanding it once makes the rest concrete.

- **R** is the radial distance from the beam centre (BC), in detector pixels. Concentric Debye–Scherrer rings from a powder sit at fixed R.
- **η** (eta) is the azimuthal angle around the beam centre (0–360°). A perfect powder ring has the *same* intensity at every η .
- **Integration** sorts every detector pixel into an (η_{bin} , R_{bin}) cell and accumulates its counts. The result is a 2-D array of shape ($n_{\text{eta_bins}}$, $n_{\text{r_bins}}$) — the "cake". Collapsing the cake along η gives the 1-D radial profile $I(R)$ (then $I(Q)$ after an axis change).

The geometry that maps a pixel $(y, z) \rightarrow (\eta, R)$ depends on the calibration (beam centre, sample-to-detector distance `Lsd`, detector tilts `tx/ty/tz`, pixel size, and a distortion polynomial). Building that mapping once is the expensive "detector map" — in the GUI this is `build_integration_context()` (`workers.py`) which wraps the backend `*.from_spec(...)` geometry builders. A `spec` (an `IntegrationSpec`) is the concrete geometry object; the GUI builds it from a calibration result via `helpers._build_spec / helpers.spec_from_geometry_file`.

Two derived quantities recur:

- **count cake** — integrate an all-ones image with the same geometry. Each cell then holds the *number of pixels* that fell into it. Used to convert summed counts to per-pixel means and to weight the azimuthal average (see §5).
- **η -collapse** — turning $(n_{\text{eta}}, n_{\text{r}})$ into $(n_{\text{r}},)$. How exactly this is done is the "azimuthal averaging method" of §5.

2. Bad-pixel masking

Goal. Produce a boolean/ `uint8` mask the size of the detector, where `1` marks a pixel to be ignored during integration (hot, dead, saturated, spikes, cosmic-ray hits, detector gaps, etc.). A masked pixel is excluded from *both* the intensity sum and the bin count, so it does not bias the per-pixel mean.

Orchestration. `MaskComputeWorker` (`workers.py`) combines several detectors by logical **OR** into one mask. It starts from the *base threshold* mask (pixels below/above simple intensity cutoffs) and OR-adds each enabled method:

```
combined = base_threshold
combined |= statistical_auto_mask    (spatial outlier [+ temporal
constancy])
combined |= cosmic_ray_mask          (temporal  $\sigma$ -clip over a frame stack)
combined |= spatial_spike_mask       (single-frame Laplacian)
```

```
combined |= azimuthal_clip_mask      (needs geometry)
combined |= learnable_mask           (needs geometry; trained)
```

A method contributes only if enabled; the geometry-based ones additionally require a calibration result. Temporal methods (statistical median, cosmic-ray) load the frame **stack** once (`_load_image` over `stack_paths`) and share it.

Each method is described below with its exact algorithm and parameters.

2.1 Statistical auto mask

Two **independently selectable** components, each backed by its own function and each OR'd in only when its checkbox is ticked (either one, or both): `spatial_outlier_mask()` (the *Spatial outlier* checkbox) and `temporal_constancy_mask()` (the *Temporal constancy* checkbox). The UI passes `stat = {"spatial": bool, "temporal": bool, k_sigma, hot_factor, dead_factor, frozen_frac, overflow}`; `MaskComputeWorker` runs `spatial_outlier_mask` iff `stat["spatial"]`, and `temporal_constancy_mask` iff `stat["temporal"]` (the latter additionally needs a stack of ≥ 2 frames, else it is skipped with a note). All $\sigma / K_\sigma / n_\sigma$ fields in the Mask tab are unbounded ($\max \approx 1e9$), so any threshold is accepted.

2.1.1 Spatial outlier detection — `spatial_outlier_mask()`

Detects pixels that are anomalously bright/dark **relative to their local neighbourhood**, using a robust (median/MAD) local statistic so real ring intensity does not trigger it. The input `med` is the per-pixel temporal median of the stack (or the single frame if no stack); `stackmax` is the per-pixel max over the stack (for saturation).

Algorithm (exact):

1. **Local baseline** — `mf = median_filter(med, size=5)`. A 5×5 median is the expected value of each pixel from its neighbours; robust to a single bad pixel.
2. **Residual** — `resid = med - mf`. How far each pixel departs from its local neighbourhood.
3. **Local robust scale** — `local_scale = median_filter(|resid|, size=15) × 1.4826 + 1e-6`. The 15×15 median of `|resid|` is the local MAD; $\times 1.4826$ converts MAD → an estimate of the local Gaussian σ . This makes the threshold *adaptive*: noisy regions get a larger scale, quiet regions a smaller one.
4. **Z-score** — `z = resid / local_scale` (local, robust standard score).
5. **Gating** (needs both a large Z *and* a real intensity departure, so shot noise on bright rings is not flagged):
 - `mf_safe = clip(mf, 1e-9, ∞)`
 - `hot = (z > k_sigma) & (med > hot_factor · mf_safe)`
 - `dead = (z < -k_sigma) & (med < dead_factor · mf_safe)`
 - `sat = stackmax ≥ overflow` (if an `overflow` value is given, e.g. the detector's uint16/uint32 saturation sentinel)
 - `mask = hot | dead | sat`

Parameters (from the Mask tab's statistical group, passed as `stat`):

- `k_sigma` — robust-Z threshold (typical ≈ 5). Higher $\rightarrow$ fewer flags.
- `hot_factor` — a hot pixel must also exceed `hot_factor × local median` (e.g. 2 $\rightarrow$ at least 2× its neighbourhood). Guards against flagging genuine sharp rings.
- `dead_factor` — a dead pixel must be below `dead_factor × local median` (e.g. 0.5).
- `overflow` — saturation sentinel; `None` disables the `sat` term.

Why local + robust? A global mean/ σ would be dominated by the huge dynamic range between rings and background, flagging entire rings as "hot". The local median baseline removes ring structure; the local MAD scales the threshold to the local noise; the intensity-ratio gate prevents Poisson fluctuations on bright rings from being mistaken for defects.

2.1.2 Temporal constancy — `temporal_constancy_mask()`

Catches pixels/modules that are **stuck at a constant value** across frames (dead module, detector gap, frozen ADC) — which a single-frame spatial test can miss because a constant patch can look locally smooth.

Algorithm:

1. `temp_std = std(stack, axis=0)` — per-pixel standard deviation over frames.
2. `ref = 75th percentile of the non-zero temp_std` — a robust reference for the "typical" temporal variation of a live pixel. Using the 75th percentile of *non-zero* values keeps the reference from being dragged down by the very dead pixels we are trying to find.
3. `frozen = temp_std < frozen_frac · ref` — a pixel varying far less than typical is flagged.

Parameter: `frozen_frac` (e.g. 0.05 $\rightarrow$ flag pixels whose temporal std is below 5 % of the typical). Requires ≥ 2 frames; `frozen_frac = 0` disables it.

Together, **statistical auto mask = spatial-outlier (bright/dark/saturated by local robust Z) and/or temporal-constancy (frozen pixels)** — each toggled independently, so you can run either alone or union both.

2.2 Spatial spike rejection

A **single-frame, geometry-free** detector of isolated sharp pixels. The GUI calls the backend `midas_integrate_v2.reject_spatial_spikes(image, n_sigma, method="laplacian")` and OR-adds the returned mask.

- **Laplacian method (default):** convolve the image with a discrete Laplacian kernel (a second-derivative / edge operator). A pixel that differs sharply from its immediate neighbours (a one-pixel spike) produces a large Laplacian response; a smoothly varying region (including a broad ring) produces a small one. Pixels whose Laplacian response exceeds `n_sigma` times the robust spread of the response are flagged.
- **Parameter:** `n_sigma` (e.g. 5). It works on one image, needs no stack and no calibration — it is purely a local spatial-frequency test.

2.3 Cosmic-ray rejection

A **temporal** outlier detector across a frame stack, via

```
midas_integrate_v2.streaming.reject_cosmic_rays(stack, n_sigma,  
mode="flag_only", use_mad=True) .
```

- For each pixel, look at its value across all frames. A cosmic-ray hit deposits a huge transient in **one** frame while the pixel is normal in the others. Take the per-pixel temporal median and the temporal **MAD** (`use_mad=True` → robust to the outlier itself); any frame-value beyond $n_sigma \times 1.4826 \cdot MAD$ from the median is a cosmic-ray hit.
- `mode="flag_only"` returns a per-frame boolean cube `cr_mask_3d` (which pixels in which frames were hit) without altering the data. The GUI collapses it with `cr_mask = cr_mask_3d.any(axis=0)` — a pixel hit in *any* frame is masked everywhere (conservative, since a static mask applies to the whole scan).
- **Requires ≥3 frames** (a median/MAD over 2 frames is meaningless); the GUI skips it with a log note otherwise. Parameter: `n_sigma` (e.g. 5).

2.4 Spatial spike vs. spatial outlier

These two sound similar but are different tools; the distinction matters when choosing which to enable.

	Spatial spike (§2.2)	Spatial outlier (§2.1.1)
Backend	<code>reject_spatial_spikes</code>	<code>spatial_outlier_mask</code> (GUI-local, scipy)
Domain	Single frame	Single frame <i>or</i> temporal median of a stack
Operator	Laplacian (2nd-derivative edge response)	Local median residual → local MAD Z-score
What it targets	Isolated 1-pixel <i>spikes</i> — sharp high-spatial-frequency points	Pixels that are anomalously hot/dead/saturated vs. their local neighbourhood, with intensity-ratio gating
Neighbourhood	Tiny (Laplacian kernel, immediate neighbours)	5×5 baseline, 15×15 scale — larger, robust
Extra gates	None (pure frequency test)	Requires both a large Z and <code>med ≥ factor · local_median</code> ; plus a saturation term
Typical catch	Zingers, single stuck-bright pixels, salt-and-pepper	Hot clusters, dead pixels, saturated pixels, whole bad regions vs. background
Cost / needs	Cheapest; no stack, no geometry	Cheap; benefits from a stack (temporal median)

In short: **spike** = "does this pixel stick out sharply from its immediate neighbours in one frame?" (a high-pass filter). **Outlier** = "is this pixel's robust local Z-score extreme *and* its intensity genuinely hot/dead/saturated?" (a robust local anomaly test with physical gating). They are complementary and can both be enabled — the spike test catches single zingers the outlier test's median baseline may absorb, while the outlier test catches sustained hot/dead regions and saturation the Laplacian ignores.

2.5 Calibration-based masks

Two methods need the geometry (a calibration result), because they reason in the (η, R) frame rather than pixel space.

2.5.1 Azimuthal σ -clip

`midas_integrate_v2.azimuthal_sigma_clip(image, geom, n_sigma)`, with a `HardBinGeometry` built from the calibration (`_build_spec(result, 2.0, 5.0)`).

- **Idea (powder isotropy):** all pixels at the same radius R (same ring) should have the same intensity. So for each ring, take the median (and robust spread) of the pixels along η ; any pixel deviating by more than `n_sigma` from its ring's median is anomalous — a bad pixel, a shadow, a streak, a single-crystal spot on a powder ring, etc.
- This catches defects that lie *on* a ring and therefore look "reasonable" in intensity (a spatial test wouldn't flag them) but break the azimuthal symmetry.
- Parameter: `n_sigma`. Needs geometry because "same ring" requires the pixel $\rightarrow$ R map.

2.5.2 Learnable mask

A differentiable, trained mask (`midas_integrate_v2.LearnableMask`) that *learns* which pixels to down-weight so the integrated cake becomes azimuthally uniform.

- A per-pixel weight in $[0,1]$ (initialised at `init_weight`, default 0.9) sits on top of the static combined mask (`static_mask=combined`).
- Training loop (Adam, `n_steps` default 300, `lr` default 0.5):

```
int2d = integrate_with_corrections(img, spec, learnable_mask=lm)
loss = EtaUniformityLoss(int2d) + sparsity_prior(lm,
weight=sparsity_weight, target=1.0)
loss.backward(); opt.step()
```

- `EtaUniformityLoss` (see §4) rewards rings that are flat in η — so the mask learns to suppress exactly the pixels that break azimuthal symmetry.
- `sparsity_prior` (weight default 1e-4, target 1.0) pulls weights toward 1 (keep pixels) so the mask stays *sparse* — it only masks pixels that genuinely help uniformity, rather than trivially masking everything.
- After training, `extract_hard_mask(threshold=0.5)` converts the soft weights to a boolean mask, OR'd into the combined result.
- Slowest method; use when the simpler detectors leave residual azimuthal artefacts. Needs geometry.

3. Calibration pipelines

Goal. From a calibrant powder pattern (CeO_2 , LaB_6 , ...) recover the detector geometry:

`Lsd`, beam centre `BC_y/BC_z`, tilts `tx/ty/tz`, optionally wavelength and a distortion polynomial (15 coefficients), plus a residual strain estimate.

The GUI's calibration layer is `midas_gui/calib.py`, which hides the fact that the backend exposes *several* pipelines with different call signatures and different result types. Two functions unify them: `run_pipeline(mode, image, dark, cfg)` dispatches, and `normalize_result(...)` converts any pipeline output into a single `AutoCalibrationResult` the rest of the GUI understands.

3.1 Common prerequisites

- **Seed** (`make_seed_safe`): the automatic seeder (`auto_seed.make_seed`) estimates an initial BC and Lsd from the ring pattern. It is always called with `use_diplib=False` — `diplib`'s median filter *segfaults* on macOS, and a native crash cannot be caught, so the safe path is mandatory. If auto-seeding fails and no manual seed is provided, the advanced pipelines raise (asking the user to Pick BC / Pick Ring + Lsd in Tab 2). A manual seed `{BC_y, BC_z, Lsd}` from the UI overrides the auto seed.
- **v1 params** (`build_v1_params`): builds a `CalibrationParams` (the LM optimiser's input) from the seed. Notable fields:
 - `RhoD = rho_px · pxY` where `rho_px` is the BC-to-farthest-corner distance in px — sets the maximum meaningful radius.
 - `MaxRingRad = 0.97 · rho_px` (default), `MinRingRad = 120 px` (default) — the radial window of rings used for the fit (drop the beam-stop region and the corners).
 - `SpaceGroup` and `LatticeConstant` from the chosen calibrant (`_SG` , `_LC`).
 - Refine flags via `_refine_dict` : the GUI's `Lsd/BC/ty/tz/tx/Wavelength/Distortion` checkboxes map to the v1 refine dict; the single **Distortion** flag sets all 15 `p0..p14` coefficients on/off together; `Parallax` is always off.

3.2 The pipelines (`mode`)

- **one_shot** (default) → `midas_calibrate_v2.calibrate(image, ...)`. The fully-automatic path: auto-seed (or manual BC+Lsd), then a Levenberg–Marquardt refine of the selected parameters in a single call. Options passed through: `n_iter`, `lm_max_iter`, `refine_tilts` (on if `ty` or `tz`), `refine_distortion`, `build_residual_corr` (optionally build a per-pixel residual-correction map), `pxZ`, `im_trans`. Best when the seed is reliable and you want a quick, good geometry. *Caveat*: on weakly-tilted data one-shot can report a spurious self-compensated tilt — prefer four-stage/first-time then.
- **first_time** → `first_time_calibrate(...)`. For an **unknown** geometry with only a coarse Lsd guess (`DEFAULT_LSD_UM` or the manual value). Takes the lattice + space group explicitly and is more tolerant of a poor starting point; slower but robust for a brand-new setup. Its result carries a `history` (per-iteration strain) whose last `mean_strain_uE` becomes the reported strain.
- **four_stage** → `autocalibrate_four_stage(v1, img, ...)`. A staged refinement that separates concerns for stability: coarse → geometry → (distortion) → strain. `normalize_result` reads `raw.stage2` (the final geometry stage) for the geometry and `raw.stage4_strain_uE` for residual strain. The most trustworthy choice for tilt/strain-sensitive work; also the route used internally when a **panel layout** is requested (see below).
- **bayesian** → `autocalibrate_bayesian(v1, img, mode="laplace", ...)`. Returns a MAP estimate (`map_unpacked`) plus a **Laplace-approximation σ per refined parameter** (`laplace.sigma_per_dim`), which the GUI stores as

`result._laplace_sigma` for display. Use when you want uncertainty estimates on the geometry.

- **joint** → `autocalibrate_joint(v1, img, ...)`. Joint refinement over the full cake/multi-ring objective; returns `map_unpacked`.

Panel layout. If the detector is a multi-panel array, `cfg["panel_layout"]` builds a `PanelLayout.regular(n_y, n_z, sy, sz, gap_y, gap_z)`. For `one_shot` with panels, the GUI deliberately reroutes through `autocalibrate_four_stage` because `calibrate()` drops the refined per-panel shifts before returning, whereas `stage2.unpacked` retains them (`panel_delta_yz/theta/lsd/p2`), which are then attached to the result for a companion `panel_shifts.txt` export.

3.3 Result normalisation

`normalize_result` maps whichever raw result came back to an `AutoCalibrationResult` with fields `Lsd`, `BC_y`, `BC_z`, `tx`, `ty`, `tz`, `distortion{...}`, `pxY`, `pxZ`, `NrPixelsY`, `NrPixelsZ`, `wavelength_A`, `post_residual_strain_uE`, plus optional `_panel_unpacked`, `_laplace_sigma`, and a residual-correction map. From here the GUI is pipeline-agnostic: spec building, ring drawing, paramtest/JSON export and the results panel all consume this one type.

4. Refinement with η -uniformity

Tab 3 (Calibration Refinement). Given an existing calibration and a real frame, nudge the geometry so the integrated rings become as **azimuthally uniform** (flat in η) as possible. Implemented in `RefinementWorker` (`workers.py`).

4.1 The loss

Integrate the frame to a cake `int2d` of shape `(n_eta, n_r)` (hard binning), then per radial bin compute the mean and variance over η :

```
m_e = mean over  $\eta$       (per R bin)  # the ring's average intensity
v_e = variance over  $\eta$   (per R bin)  # how much it wobbles around the
ring
w   = clip(m_e, 0,  $\infty$ )           # intensity weight (bright rings
matter more)
eta_loss =  $\sum_R (v_e \cdot w) / \sum_R (w^2)$ 
```

A geometrically correct calibration makes every ring flat → `v_e` → 0 → `eta_loss` → 0. Weighting by `w` (and normalising by $\sum w^2$) makes strong rings dominate and keeps the loss scale-invariant. The image is first normalised to $O(1)$ (`img / mean(img>0)`) so the loss is well-conditioned. This same `EtaUniformityLoss` is what the **learnable mask** (§2.5.2) minimises.

4.2 The optimiser

Refinement uses **derivative-free Nelder–Mead** (`scipy.optimize.minimize`), not autograd, because hard-bin (floor) pixel assignment makes the loss a non-differentiable staircase. To keep the simplex well-scaled, each parameter is optimised in **normalised step units**: `value = base + x · STEP`, with

```
STEP = {Lsd: 500 μm, BC_y: 0.5 px, BC_z: 0.5 px,  
        ty: 0.1°, tz: 0.1°, tx: 0.1°, Wavelength: 1e-4 Å}
```

so every coordinate `x` is $O(1)$. A **symmetric initial simplex** explores both $\pm$ directions. `maxiter ≥ 400`, `xatol=1e-3`, `fatol=1e-5`.

4.3 Guards and the beam-centre subtlety

- **Hard bounds** `MAX_STEPS = 3` (± 3 normalised units $\rightarrow$ BC ± 1.5 px, tilt $\pm 0.3^\circ$, Lsd $\pm 1500 \mu\text{m}$, $\lambda \pm 3e-4 \text{ Å}$). Outside this the objective returns a steep penalty without touching the spec, keeping the search physical.
- **Degenerate-minimum guard**: if the weighted denominator $\sum w^2$ collapses ($< 1e-4$) the rings have been pushed outside the integration R-range, giving a fake "perfectly uniform" (empty) cake — treated as a penalty, not a minimum.
- **BC weak-sensitivity + L2 anchor**: η -uniformity barely responds to the beam centre — shifting BC *translates* rings but keeps them circular (near-zero azimuthal-variance signal), whereas tilts deform rings into ellipses (large, unambiguous signal). So BC uses a small step (0.5 px) and a soft L2 anchor `bc_reg = f0 · 2e-3 ·  $\sum x_{bc}^2$` that prevents aimless BC drift across the flat landscape while still allowing a correction when the signal clearly exceeds the regularisation cost. Large BC errors should be fixed by Tab 2 calibration, not here.
- **Revert-if-worse**: if the optimised loss exceeds `1.05 × f0`, the original geometry is returned unchanged. NaNs revert to the last good value.
- **Mask handling**: the mask tensor is passed to the geometry builder every iteration so masked pixels are excluded from *both* the intensity sum and the bin count. Merely zeroing them in the image is not enough — `HardBinGeometry` would still count them, dragging the mean down and inflating η -variance.

Only parameters that are actual refinable tensors on the spec are optimised; the final values are written back into a copy of the calibration result.

5. Azimuthal averaging methods & binning kernels

Tab 4 (Batch Integrate). Two orthogonal choices control how a 2-D frame becomes a 1-D profile: the **binning kernel** (how pixels are distributed into `( $\eta$ , R)` cells) and the **azimuthal averaging** (how the cake is collapsed over η). Both live in `integrate_frame()` (`workers.py`).

5.1 Binning kernels (pixel → cell assignment)

Selected by the Kernel dropdown; geometry built by `build_geom(spec, kernel, mask)`:

- **Hard** (`HardBinGeometry`, `integrate_hard`): each pixel is assigned wholly to the single (η, R) bin its centre falls in (floor assignment). Fastest; introduces small binning/aliasing steps (this is also why refinement uses a derivative-free optimiser — see §4.2).
- **Subpixel K=2 / K=4** (`SubpixelBinGeometry`, `integrate_subpixel`): each pixel is split into a $K \times K$ grid of sub-pixels, each assigned independently. This smooths the assignment and reduces aliasing at bin boundaries; $K=4$ is smoother and $\sim 4\times$ the work of $K=2$.
- **Polygon** (`PolygonBinGeometry`, `integrate_polygon`): computes the **exact geometric overlap area** between each pixel's polygon and each bin, and distributes counts by area fraction. The most accurate (no assignment discretisation) but the geometry build is **slow — minutes for a 2880^2 detector** — so cache it or use hard/subpixel for interactivity.

The kernel only changes *how counts land in cells*; the cake shape (n_{η}, n_r) is the same.

5.2 Azimuthal averaging (cake → 1-D), `_profile_from_cake()`

Once you have the cake, collapse over η . Two methods (the **Azim. avg** dropdown):

- **η -bin mean (legacy)**: `prof = nanmean(cake, axis=0)` — the unweighted mean of the per- η -bin means, each η bin counting equally. Simple, but when the beam centre is off the detector or azimuthal coverage is partial, some η bins are filled by very few pixels yet still count as much as fully-populated bins, biasing the profile (worse with a coarse η bin).
- **Pixel-weighted (default)**: using the **count cake** (pixels per cell),

$$\text{prof}(R) = \sum_{\eta} (\text{cell_mean}(\eta, R) \cdot \text{count}(\eta, R)) / \sum_{\eta} \text{count}(\eta, R)$$

i.e. each η bin contributes in proportion to how many real pixels it holds. This is independent of the η -bin size and robust to partial/uneven azimuthal coverage and off-detector beam centres. It reduces to the plain mean when coverage is uniform.

5.3 Variance / σ , corrections, and the routing in `integrate_frame`

`integrate_frame` picks one of three paths (mutually exclusive):

1. **Corrections enabled** → `integrate_with_corrections(img, spec, polarization, solid_angle)`. This returns **summed** (un-normalised) counts per cell, so it is divided by the **count cake** (`corrections_counts`, the same function on an all-ones image) to recover the per-pixel mean. σ falls back to $\sqrt{I}$. (Corrections win over variance — see below.)
2. **Variance enabled** → `integrate_<kernel>_with_variance(img, geom, error_model)`. Returns per-cell mean *and* per-cell σ from the chosen error model

(poisson / azimuthal / hybrid). The σ of the η -collapsed profile is propagated as $\sqrt{\sum \sigma^2 / N}$ over the valid η bins.

3. **Plain** → `integrate_<kernel>(img, geom, normalize=True)` ; $\sigma = \sqrt{I}$.

In all three, if pixel-weighting is on, the η -collapse uses the count cake (`cnt_cake` for plain/variance, `corr_counts` for the corrections path).

Corrections and variance are **mutually exclusive**: if both are requested the GUI disables variance and logs a note ($\sigma = \sqrt{I}$), because the corrections path does not carry an error model.

5.4 Q-uniform binning

The kernels bin uniformly in **R (px)**, not Q. When "Q-uniform" is requested the worker integrates R-uniform as usual, then **rebins onto a uniform-Q grid** by interpolation (`rebin_R_to_Q`): $Q = 4\pi \sin(\theta)/\lambda$ with $\theta = \frac{1}{2} \cdot \text{atan}(R \cdot \text{px} / \text{Lsd})$, sort by Q, `np.interp` the profile and σ onto the requested [Qmin, Qmax, ΔQ] grid. Done this way (rather than native Q-binning) because the kernels lack a Q-mode; interpolating after R-integration puts rings at the correct Q.

6. Physics corrections

Two pixel-domain corrections (Tab 4, `CorrectionFlagsWidget.build_corrections`), applied inside `integrate_with_corrections` :

- **Polarization** (`PolarizationCorrection(pol_fraction, pol_plane_eta_deg)`): a synchrotron beam is (nearly) linearly polarized in the horizontal plane, so the Thomson scattering cross-section — and hence the measured intensity — varies with both scattering angle 2θ and azimuth η . The correction divides each pixel by its polarization factor so rings are not artificially brightened/dimmed around the azimuth. Parameters: `pol_fraction` (degree of horizontal polarization, default 0.99) and `pol_plane_eta_deg` (orientation of the polarization plane in the η frame, default 0°).
- **Solid angle, tilt-aware** (`SolidAngleCorrection()`): a flat detector subtends a *smaller* solid angle per pixel at larger scattering angles (and asymmetrically when the detector is tilted). Pixels farther from the beam therefore collect fewer photons purely for geometric reasons. The correction divides by the per-pixel solid angle (accounting for `tx/ty/tz` tilt), removing that geometric fall-off so the profile reflects true scattering power.

Mechanics. Because `integrate_with_corrections` returns *summed* counts, the GUI normalises by the count cake (§5.3) to get per-pixel means comparable to the plain kernels. Enabling either correction forces $\sigma = \sqrt{I}$ (no error model on this path). Both are optional and independent.

7. Monitor normalisation

Corrects for shot-to-shot variation in **incident flux** (ring current, shutter jitter, beam decay) so profiles from different frames are on a common scale.

- Input: a text file (one floating-point value per line), one value per **processed** frame — parsed in `BatchWorker.run` as `monitor_vals`.
- For each processed frame the profile and its σ are divided by the corresponding scalar:

```
prof  = prof  / mon
sigma = sigma / |mon|
cake  = cake  / mon      (if a 2-D cake is being written)
```

- Indexing uses `proc_idx`, a counter that advances **only for frames that pass the frame-range/stride filter**, so monitor line k pairs with the k -th processed frame (not the k -th file on disk). A zero monitor value is skipped (no division). This is distinct from the live-folder **MONITOR** button, which is unrelated (that watches for new files; see `gui_documentation.md`).

8. Drift correction

For **long scans** where the geometry slowly drifts (thermal expansion of the stage, sample-height changes) so a single calibration is not valid for every frame. Implemented by `DriftWorker + _spec_from_trajectory (workers.py)`, wrapping `midas_integrate_v2.pipelines.drift.fit_drift_trajectory`.

8.1 Fitting the trajectory

- **anchors**: a JSON of known-good geometry at specific frame indices — `{frame_idx: {"Lsd":..., "BC_y":..., "BC_z":...}}` — typically from calibrant exposures interleaved through the scan. At least 2 are required.
- `fit_drift_trajectory(anchors, sample_indices, spec, parametrization, n_knots, bayesian_sigma)` fits smooth functions `Lsd(t)`, `BC_y(t)`, `BC_z(t)` over the frame index `t`:
 - `parametrization`: `spline` (B-spline, smooth; default), `linear` (piecewise), or `constant`.
 - `n_knots`: spline flexibility (default 5) — more knots track faster drift but risk overfitting few anchors.
 - `bayesian_sigma`: also return a Laplace-approx uncertainty band on the fitted curves.
- The result is a `DriftTrajectory` (`frame_indices`, `Lsd_t`, `BC_y_t`, `BC_z_t`, and optional σ), reported in the UI as the fitted Lsd range.

8.2 Applying it per frame

During batch integration with drift enabled, each frame gets its **own geometry**: `_spec_from_trajectory(base_spec, traj, frame_abs_idx)` deep-copies the base spec and overwrites `Lsd/BC_y/BC_z` with values **linearly interpolated** from the trajectory at that absolute frame index. Because the geometry changes per frame, the binning geometry (and count cakes) are **rebuilt per frame** in the drift path of `BatchWorker` — more expensive than the single-geometry fast path, but necessary for correctness. Tilts and distortion are taken from the base calibration (only Lsd/BC are treated as drifting).

End of implementation details. If a formula or default here ever disagrees with the code, the code in `midas_gui/workers.py` / `calib.py` / `widgets.py` is authoritative — update this document to match.